91da7674-7722-4724-90ca-7821f737ae

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw/gh-aw (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-26T02:58:35.954+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
feat: add `replay` command for rendering unified timeline logs
</pr_title>

<pr_description>
Adds `gh aw replay <run-id-or-url>` — a command that downloads run artifacts and renders a merged, chronological activity timeline to the console, simulating what you'd see watching a Copilot CLI session live.

What's new

- **`pkg/cli/replay_command.go`** — `NewReplayCommand` / `ReplayWorkflowRun`; accepts the same run ID/URL formats as `audit`, downloads via the existing `downloadRunArtifacts` helper, then feeds the local log dir into `BuildUnifiedTimeline` + `renderUnifiedTimeline`

- **`cmd/gh-aw/main.go`** — registers `replay` in the `"analysis"` group alongside `logs` and `audit`
- **`pkg/cli/replay_command_test.go`** — command structure, stdout output assertions (verifies `agent_turn`, `tool_start`, `tool_done`, server/tool labels present), and empty-dir graceful-warning path

Example output

```
gh aw replay 12345678

i Unified MCP + Firewall + Agent Event Timeline

Total Events  : 12
  Gateway     : 4  (tool_calls=4, difc_filtered=0, guard_blocked=0)
  Firewall    : 5  (allowed=4, blocked=1)
  Agent       : 3  (turns=1, tool_start=1, tool_done=1)

Event Timeline
┌─────────────┬───┬──────────────┬──────────────────────┬─────────┐
│Time         │Src│Kind          │Detail                │Status   │
├─────────────┼───┼──────────────┼──────────────────────┼─────────┤
│10:01:02.000│AG │💬 agent_turn │turn 1                │         │
│10:01:02.120│AG │▶ tool_start  │github/search_code    │         │
│10:01:02.140│GW │🔧 tool_call  │github/search_code    │142ms    │
│10:01:02.145│FW │✓ net_allowed │api.github.com:443    │CONNECT  │
│10:01:02.300│AG │■ tool_done   │github/search_code    │success  │
└─────────────┴───┴──────────────┴──────────────────────┴─────────┘
```

Artifacts are cached in the same directory layout as `logs`/`audit`; re-running against the same run ID reads from disk.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@pelikhan</author>
@copilot no stats or table, render like an agentic engine
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot no stats or table, render like an agentic engine

Done in the latest commit. `replay` now uses `renderUnifiedTimelineStream` — no stats header, no table. Agent turns render as `> Turn N  [HH:MM:SS]` section headers, with tool starts/dones and gateway/network events indented beneath them using the existing icons:

```
> Turn 1  [10:01:02]
  ▶ github/search
    🔧 github/search  142ms
    ✓ api.github.com:443  CONNECT
  ■ github/search  success
```

The existing `renderUnifiedTimeline` (table + stats) is unchanged and still used by `audit` / `displayUnifiedTimeline`.
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot apply colors using existing helpers and palette to make it more delightful
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot should show the first few lines of user/assistant/reasoning messages
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot apply colors using existing helpers and palette to make it more delightful

Done in the latest commit. Colors are now applied via a TTY-guarded `streamColor` helper using the existing `styles` palette:

- **Turn headers** — bold purple (`styles.Command`) with muted timestamp (`styles.LineNumber`)
- **Tool start** `▶` — yellow (`styles.Progress`)
- **Tool done** `■` + status — green (`styles.Success`) on success, red (`styles.Error`) on error
- **Gateway tool call** `🔧` — bold purple (`styles.ServerName`); duration muted, error annotation red
- **Network allowed** `✓` — cyan (`styles.Info`); HTTP method muted
- **Network blocked** `✗` + `[blocked]` — red (`styles.Error`); HTTP method muted
- **DIFC filtered** `⊘` + reason — orange (`styles.Warning`)
- **Guard blocked** `🛡` + annotation — red (`styles.Error`)

Colors are suppressed automatically when stdout is not a TTY, keeping piped output clean.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot should show the first few lines of user/assistant/reasoning messages

Done. The replay stream now shows the first few lines (up to 3) of message content for user, assistant, and reasoning events.

What changed:
- `user.message` → the user's prompt is shown indented below the `> Turn N [HH:MM:SS]` header (muted `ContextLine` style)
- `assistant.message` → new event kind `assistant_message` (🤖 icon), content shown indented in `ContextLine` style
- `reasoning` / `assistant.reasoning` → new event kind `reasoning` (💭 icon), content shown indented in `Verbose` (muted) style
- Content is capped at `streamMaxMessageLines = 3` lines, each capped at `streamMaxLineLength = 80` runes; a `…` marker is appended when truncated
- Colors are suppressed when stdout is not a TTY (piped output stays clean)

````
Example output:
```
> Turn 1  [10:01:02]
  What files are in the repo?
  💬
  I should check the repository structure first.
  🤖
  Here are the files I found in the repository:
  README.md, main.go, …
  ▶ github/list_files
    🔧 github/list_files  85ms
  ■ github/list_files  success
```
</comment_old>

  <comment_new>
  <comment_id>4539508379</comment_id>
  <author>@pelikhan</author>
  @copilot use Unicode cross compatible symbols instead of
  emojis. Reuse existing ones used in step summaries.
  </comment_new>

</pr_comments>

</comments>

---
The last **7** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a suggestion.
```

- Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

## <prev_assistant>

Task completed.

## <current>

### <request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw/gh-aw (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-26T03:32:34.433+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

---

````
*This section includes details on the pull request with code
changes you have already submitted to fix the problem above.*

<pr_title>
feat: add `replay` command for rendering unified timeline logs
</pr_title>

<pr_description>
Adds `gh aw replay <run-id-or-url>` — a command that downloads
run artifacts and renders a merged, chronological activity
timeline to the console, simulating what you'd see watching a
Copilot CLI session live.

What's new

- **`pkg/cli/replay_command.go`** — `NewReplayCommand` /
`ReplayWorkflowRun`; accepts the same run ID/URL formats as
`audit`, downloads via the existing `downloadRunArtifacts`
helper, then feeds the local log dir into `BuildUnifiedTimeline`
+ `renderUnifiedTimeline`
- **`cmd/gh-aw/main.go`** — registers `replay` in the
`"analysis"` group alongside `logs` and `audit`
- **`pkg/cli/replay_command_test.go`** — command structure,
stdout output assertions (verifies `agent_turn`, `tool_start`,
`tool_done`, server/tool labels present), and empty-dir
graceful-warning path

Example output

```
gh aw replay 12345678

i Unified MCP + Firewall + Agent Event Timeline

Total Events  : 12
  Gateway     : 4  (tool_calls=4, difc_filtered=0,
guard_blocked=0)
  Firewall    : 5  (allowed=4, blocked=1)
  Agent       : 3  (turns=1, tool_start=1, tool_done=1)

Event Timeline
┌─────────────┬───┬──────────────┬──────────────────────────┬──────────
┐
│Time         │Src│Kind          │Detail                    │Status
│
├─────────────┼───┼──────────────┼──────────────────────────┼──────────
┤
````
````

```
|10:01:02.000|AG |💬 agent_turn |turn 1                |
|
|10:01:02.120|AG |▶ tool_start  |github/search_code    |
|
|10:01:02.140|GW |🔧 tool_call  |github/search_code    |142ms
|
|10:01:02.145|FW |✓ net_allowed |api.github.com:443    |CONNECT
|
|10:01:02.300|AG |■ tool_done   |github/search_code    |success
|
└────────────┴──┴──────────────┴──────────────────────┴────────
┘
```

Artifacts are cached in the same directory layout as `logs`/`audit`; re-running against the same run ID reads from disk.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@pelikhan</author>
@copilot no stats or table, render like an agentic engine
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot no stats or table, render like an agentic engine

Done in the latest commit. `replay` now uses `renderUnifiedTimelineStream` — no stats header, no table. Agent turns render as `> Turn N  [HH:MM:SS]` section headers, with tool starts/dones and gateway/network events indented beneath them using the existing icons:

```
> Turn 1  [10:01:02]
  ▶ github/search
    🔧 github/search  142ms
    ✓ api.github.com:443  CONNECT
  ■ github/search  success
```

```
The existing `renderUnifiedTimeline` (table + stats) is
unchanged and still used by `audit` / `displayUnifiedTimeline`.
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot apply colors using existing helpers and palette to make
it more delightful
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot should show the first few lines of user/assistant/
reasoning messages
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot apply colors using existing helpers and palette to
make it more delightful

Done in the latest commit. Colors are now applied via a TTY-
guarded `streamColor` helper using the existing `styles`
palette:

- **Turn headers** — bold purple (`styles.Command`) with muted
timestamp (`styles.LineNumber`)
- **Tool start** `▶` — yellow (`styles.Progress`)
- **Tool done** `■` + status — green (`styles.Success`) on
success, red (`styles.Error`) on error
- **Gateway tool call** `🔧` — bold purple
(`styles.ServerName`); duration muted, error annotation red
- **Network allowed** `✓` — cyan (`styles.Info`); HTTP method
muted
- **Network blocked** `✗` + `[blocked]` — red (`styles.Error`);
HTTP method muted
- **DIFC filtered** `⊘` + reason — orange (`styles.Warning`)
- **Guard blocked** `🛡` + annotation — red (`styles.Error`)

Colors are suppressed automatically when stdout is not a TTY,
keeping piped output clean.
</comment_old>

<comment_old>
<author>@copilot</author>
```

> @copilot should show the first few lines of user/assistant/reasoning messages

Done. The replay stream now shows the first few lines (up to 3) of message content for user, assistant, and reasoning events.

What changed:
- `user.message` → the user's prompt is shown indented below the `> Turn N [HH:MM:SS]` header (muted `ContextLine` style)
- `assistant.message` → new event kind `assistant_message` (🤖 icon), content shown indented in `ContextLine` style
- `reasoning` / `assistant.reasoning` → new event kind `reasoning` (💬 icon), content shown indented in `Verbose` (muted) style
- Content is capped at `streamMaxMessageLines = 3` lines, each capped at `streamMaxLineLength = 80` runes; a `…` marker is appended when truncated
- Colors are suppressed when stdout is not a TTY (piped output stays clean)

Example output:
```
> Turn 1  [10:01:02]
  What files are in the repo?
  💬
  I should check the repository structure first.
  🤖
  Here are the files I found in the repository:
  README.md, main.go, …
  ▶ github/list_files
    🔧 github/list_files  85ms
  ■ github/list_files  success
```
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot use Unicode cross compatible symbols instead of emojis. Reuse existing ones used in step summaries.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot use Unicode cross compatible symbols instead of emojis. Reuse existing ones used in step summaries.

Done in the latest commit. All emojis in `timelineEventIcon` are replaced with cross-compatible Unicode symbols, reusing the existing step-summary convention where applicable:

Event	Before	After
`tool_call`	🔧	`⚙` (gear)
`difc_filtered`	🚫	`⊖` (circled minus)
`guard_blocked`	🛡	`⊗` (circled X)
`agent_turn`	💬	`○` (open circle)
`assistant_message`	🤖	`●` (filled circle)
`reasoning`	💭	`◐` (half-filled circle — matches the `◐` prefix used for thinking content in step summaries)

The already-Unicode symbols (`✓`, `✗`, `▶`, `■`, `·`) are unchanged.
</comment_old>

<comment_new>
<comment_id>4539724573</comment_id>
<author>@pelikhan</author>
@copilot rename command "replay" to "view" since it is not rerunning the safe outputs, just rendering. Add section to also show safeoutputs in friendly manner
Finish with html url to action workflow run page
</comment_new>

</pr_comments>

</comments>

The last **8** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. Treat <comment_old> comments as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you (@copilot). To understand the context of comments that explicitly mention you, you may use other comments that do not explicitly mention you. Do not act on comments that don't explicitly mention you or reply to them, even if they are marked as <comment_new>.

```
* If the PR author has pushed back on or rejected a suggestion in a
thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a
suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




<system_reminder>

</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement,
make sure to follow your instructions for addressing CI/Build
Failures.
```